

Chapter 1

Mission Bus Integration Lab (MBIL)

1.1 Project purpose

Mission Bus Integration Lab (MBIL) is a deterministic, configuration-driven software system that simulates a simplified aerospace mission environment built around a central data bus, redundant mission computers, subsystem messaging, defined failure modes, observability, and failover behavior.

The goal is not to make a flashy simulator

The goal to demonstrate that I understand how **mission-critical systems behave, fail, recover, and get debugged.**

1.2 What MBIL should communicate

When someone sees this project, they should think:

- ★ This person understands reliable system design.
- This person thinks in interfaces and contracts
- This person knows how to model faults and degraded operation
- This person knows how to make systems observable
- This person can separate control logic, transport, config, and monitoring

MBIL should read like a **systems engineering lab**, not a game and not a toy simulation

1.3 Core project statement

MBIL is a **tick-driven mission-system simulation platform** with these defining properties:

- Deterministic execution
 - Configuration-defined topology
 - Redundant control nodes
 - First-class failure scenarios
 - structured observability
 - Recoverable degraded operation
-

1.4 Design priorities

These are the real priorities, in order:

Priority 1 – Determinism

The same scenario should produce the same behavior every run unless randomness is explicitly enabled

Priority 2 – Reliability modeling

Failures are not side noes. They are part of the system design.

Priority 3 – Observability

Every important decision, state transition, and message path should be visible in logs or snapshots.

Priority 4 – Clear modular architecture

Subsystem logic, bus behavior, timing, fault injection, and login must stay cleanly separated.

Priority 5 – Configurability

System topology and scenarios should be changel from config files, not by rewriting core logic.

1.5 Function Goals

MBIL must support:

- Subsystem-to-subsystem communication through a central bus
 - Multiple sybssystem types
 - Dual mission computer architecture
 - Heartbeat monitoring
 - Failover behavior
 - Defined failure injection
 - Message routing
 - Deterministic Scheduling
 - Structured logs
 - State Snapshots
 - Scenario-based runs
-

1.5.1 High-level Architecture

MBIL should be split into these major layers.

A. Simulation Core

- Tick loop
- Execution order
- Simulation Duration
- System lifecycle
- Calling each subsystem in deterministic order

B. Bus Layer

- Receiving Outgoing messages
- Storing pending messages
- Applying routing rules
- Applying message-related faults
- Delivering ready messages
- Preserving deterministic ordering

C. Subsystem Layer

- Mission Computers
- Sensors
- Display / Endpoints
- Health State
- Behavior per tick
- Message Handling

D. Fault Management Layer

- Defined Failure modes
- Scripted Scenario Events
- Optional Random Fault behavior later
- Recording Fault Activations
- Applying Degradation Cleanly

E. Observability Layer

- Structured Event Logging
- Traceable Message Lifecycle
- State Snapshots
- Failover Reportin
- Auditability

F. Configuration Layer

- Loading JSON config
- Validating Topology
- Instantiating Subsystem Objects
- Loading Scenario/Fault Definitions
- Setting timing and thresholds

1.5.2 Runtime Model

MBIL should use a **fixed tick** simulation model.

- One tick every 100 ms simulated time
- Run for 300 ticks per scenario
- Deterministic Behavior
- Clean Testability
- Easier Fault scripting
- Easier replay and debugging

Tick Processing Order

Use a Fixed order every tick:

Phase 1 – Scenario updates

- Activate any scheduled Failures
- Deactivate expired Failures
- Apply Manual state changes if configured

Phase 2 – Sensor Updates

- Sensors decide whether to emit new data this tick